

Stop When Memory Suffices: Evidence-Conditioned Progressive Execution for LLM Agents

Yidan Lin, Kaixiang Wang, Jiong Lou, Jie Li

Abstract

The continued development of LLMs toward persistent and adaptive intelligence increasingly requires long-term memory mechanisms that preserve and reuse information across interactions. Existing memory systems either compress and structure histories for efficient access or perform deep research over broader trajectories. The former lowers online cost but may omit temporal, causal, or cross-step dependencies, while the latter improves evidence coverage at substantial latency and inference cost. This raises a key question: can a memory system achieve strong answer quality while maintaining low online latency? We introduce Router-Mem, an evidence-conditioned progressive execution framework for long-horizon agent memory. Router-Mem first applies a shared low-cost retrieval prefix to obtain evidence. A lightweight sufficiency router then predicts whether the context supports early termination, which enable a single-token decision at inference time. It is trained with evidence-level supervision and rationale-conditioned representation distillation. When evidence is insufficient, Router-Mem reuses retrieval hits to expand memory blocks and perform deeper analysis and aggregation. Experiments on AMA-Bench and BEAM show that Router-Mem achieves 55.17% and 38.77% score while reducing average inference time by 27.3% and 25.5% compared with full memory execution.

Introduction

Large language models are evolving from static generators into interactive systems that reason, use tools, learn from feedback, and operate over extended tasks (Yao et al. 2023; Schick et al. 2023; Shinn et al. 2023). This creates a growing need to retain and reuse information beyond a single context window (Wang et al. 2024; Zhou et al. 2024). Long-running applications accumulate conversations, tool calls, observations, errors, and intermediate decisions (Liu et al. 2024b; Yang et al. 2024). Later queries must recover relevant evidence while preserving temporal and cross-step dependencies (Li et al. 2024; Guo et al. 2024a). Although recurrent memory, sparse attention, and efficient exact attention have extended context capacity (Dai et al. 2019; Beltagy, Peters, and Cohan 2020), full-history processing remains costly (Zaheer et al. 2020; Dao et al. 2022), and performance is still sensitive to evidence position, sequence length, and task complexity (Liu et al. 2024a; Bai et al. 2024). These limitations motivate dedicated memory mechanisms for selectively

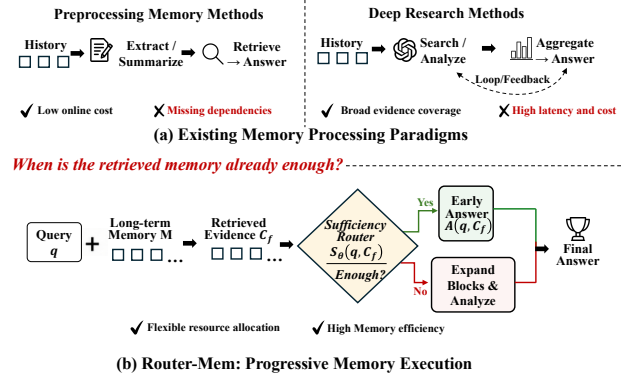


Figure 1: The Existing Memory Paradigms VS Router-mem.

retrieving and organizing query-relevant experience (Hsieh et al. 2024; Zhang et al. 2024).

Existing agent memory systems broadly follow two processing paradigms. Preprocessing-oriented methods extract, summarize, or restructure interaction histories into compact representations for efficient access (Zhang et al. 2025b; Hu et al. 2025)(as shown in Fig. 1a). Mem0 (Chhikara et al. 2025) consolidates salient conversational facts, A-MEM (Xu et al. 2025) organizes memories as dynamically linked notes, and G-Memory (Zhang et al. 2025a) builds hierarchical graphs for multi-agent histories. These designs reduce on-line retrieval cost, but early compression may omit temporal, causal, or cross-step details needed by future queries. Query-time reconstruction methods instead retain broader histories and perform iterative search and reasoning after a query arrives. GAM (Yan et al. 2025) uses a researcher agent to construct query-specific context, while E-mem (Wang et al. 2026) activates episodic segments and aggregates analyses from multiple memory agents. They improve evidence coverage, but can impose unnecessary latency when local memory already suffices, exposing a fundamental quality–efficiency trade-off (Du 2026).

However, memory-query processing need not follow a fixed paradigm. Queries whose supporting evidence is already localized should not incur the cost of deep memory research, whereas queries with incomplete or dispersed evidence may not be answered reliably through compact re-

trieval alone. This raises a central question: can a memory system infer the required processing depth from the query and the evidence recovered by lightweight retrieval, thereby allocating computation more effectively?

To address this gap, we propose Router-Mem, an evidence-conditioned progressive execution framework for long-horizon agent memory (as shown in Fig. 1b). For every query, Router-Mem first runs a shared low-cost retrieval prefix to obtain an initial evidence context. This prefix is shared by both execution paths and provides the key evidence for deciding whether the system can return early. A lightweight sufficiency router then evaluates the query together with the retrieved context and predicts whether the current evidence supports early termination. The router is trained with evidence-level supervision and rationale distillation, which transfers complex sufficiency judgments into a single-token decision at inference time. When the retrieved evidence is sufficient, the system answers directly and skips the remaining computation. Otherwise, it reuses the retrieval hits as localization signals, expands the relevant memory blocks, and invokes deeper analysis and evidence aggregation. The fast and slow paths are coupled through the shared prefix, which avoids restarting memory search from scratch. This design preserves inexpensive access for locally answerable queries while allocating costly memory reasoning to cases that still lack decisive evidence, yielding a better balance between answer quality, latency, and inference cost.

Experiments on AMA (Zhao et al. 2026) and BEAM (Tavakoli et al. 2026) show that Router-Mem achieves 55.17% and 38.77% score, while reducing average inference time by 27.3% and 25.5% relative to full memory processing. Across different routing thresholds, it consistently provides favorable quality–latency trade-offs and forms a strong empirical Pareto frontier among compared methods.

Our main contributions are as follows:

- We formulate long agentic memory based query as an evidence-conditioned progressive execution problem. The required processing depth is determined by the query and the evidence recovered at runtime.
- We propose Router-Mem, an evidence-conditioned progressive framework that addresses fixed computation in long-horizon memory access. It combines a reusable retrieval prefix, evidence-sufficiency routing, and retrieval-guided continuation to stop when evidence is sufficient and invoke broader memory analysis only when needed, balancing efficiency with deep memory reasoning quality.
- We evaluate Router-Mem on AMA-Bench and BEAM across different routing thresholds and model settings. The results demonstrate strong answer quality with substantially lower inference time and establish a favorable quality–latency Pareto frontier.

Related Work

Retrieval-Augmented Generation. Retrieval-augmented generation (RAG) grounds LLMs in external knowledge and has become a standard paradigm for knowledge-intensive tasks (Lewis et al. 2020). Some works like Adaptive-RAG,

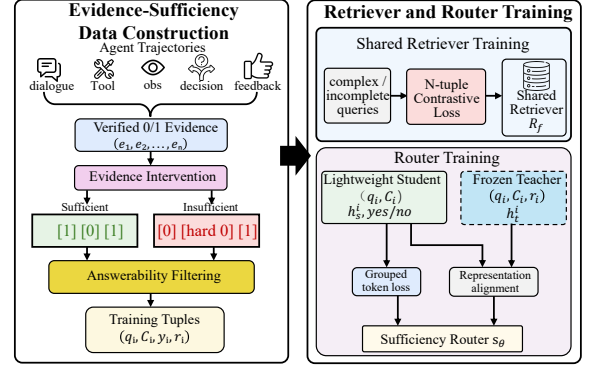


Figure 2: Training Process of Router-mem.

Stop-RAG, further selects among no retrieval, single-step retrieval, and iterative retrieval according to predicted query complexity (Jeong et al. 2024; Park, Cho, and Lee 2025; Li, Yan, and Käfer 2026). However, this query-level routing does not account for the evidence actually recovered at runtime: a difficult query may be resolved by successful initial retrieval, whereas a seemingly simple query may still lack decisive evidence. Recent work also improves retrieval through structured representations. HippoRAG combines knowledge graphs with Personalized PageRank for associative and multi-hop retrieval (Gutiérrez et al. 2024), while GraphRAG and LightRAG use graph-based indexing for global and hierarchical knowledge discovery (Edge et al. 2024; Guo et al. 2024b). Despite improving retrieval planning and evidence organization, these methods do not directly determine whether the currently retrieved evidence is sufficient to terminate further computation.

Memory Systems for LLM Agents. Long-term agent memory systems aim to efficiently preserve and access past experiences. Preprocessing-based approaches construct compact memory representations before queries arrive. Mem0 extracts and consolidates salient conversational information (Chhikara et al. 2025), A-MEM organizes experiences as evolving structured notes (Xu et al. 2025), and G-Memory builds hierarchical graphs for multi-agent interactions (Zhang et al. 2025a). ReasoningBank further distills reusable reasoning strategies from agent trajectories (Ouyang et al. 2026). These methods reduce retrieval cost but may lose fine-grained temporal and causal dependencies. In contrast, query-time reconstruction methods retain broader histories and perform deeper reasoning when queries arrive. GAM employs researcher agents to construct query-specific contexts (Yan et al. 2025), while E-mem uses episodic retrieval and multi-agent analysis for context reconstruction (Wang et al. 2026). Although these methods improve evidence coverage, they may introduce unnecessary computation when lightweight retrieval already provides sufficient evidence.

Our work connects these two regimes through a shared retrieval prefix and an evidence-sufficiency router, which terminates early when the current context supports an answer and preserves deeper processing for queries that require it.

Method

In this section we systematically presents the design of Router-Mem including training process and system pipeline. More details can be found in **technical supplement**.

Progressive Memory Execution

Long-horizon agents continuously accumulate heterogeneous memories, including conversations, intermediate decisions, tool calls, environment observations, and execution feedback. Queries over such memories require different levels of processing. Some queries can be answered from a small set of locally relevant memories recovered by lightweight retrieval. Others depend on evidence that is dispersed across time, execution steps, or interaction trajectories, and therefore require broader memory expansion and aggregation. Applying only lightweight retrieval to every query may miss such dependencies, whereas executing the full memory-processing pipeline for every query introduces unnecessary latency and inference cost.

We therefore organize memory access as a progressive execution process. Every query first performs a shared low-cost retrieval prefix. If the retrieved context already contains sufficient evidence, the system directly generates an answer and skips the remaining computation. Otherwise, it continues with broader memory processing. The fast and slow paths are coupled through the shared retrieval prefix, as the expensive stage continues from the retrieved evidence and reuses its localization signals for broader memory processing.

Determining whether the retrieved context is sufficient is itself non-trivial. A direct solution is to prompt a general-purpose LLM to inspect the query and retrieved memories and explicitly reason about whether further processing is needed. However, a reliable judgment may require checking for missing evidence, temporal updates, action–feedback relations, and conflicting records. Although the final output is only a binary decision, the underlying reasoning can be complex. Performing such deliberation online often requires a capable model, a long context prefill, and additional reasoning tokens. This cost is undesirable because the router is introduced precisely to avoid unnecessary computation. An expensive routing decision may substantially reduce the efficiency gain obtained from early termination.

To address this issue, we train a lightweight evidence-sufficiency router that directly predicts whether the current retrieval evidence state supports early termination. Given a query q and an ordered memory $M = (z_1, \dots, z_T)$, the shared retriever first returns

$$C_f = R_f(q, M), \quad (1)$$

where C_f is the initially retrieved memory context. We define the context-dependent termination label as

$$y(q, C_f) = \mathbf{1} [C_f \text{ provides sufficient evidence to answer } q], \quad (2)$$

and train the router to estimate

$$s_\theta(q, C_f) = P_\theta(y = 1 \mid q, C_f). \quad (3)$$

Importantly, the decision is conditioned on the execution state (q, C_f) rather than on the query alone. The same query may terminate after one retrieval result but require further processing after another result that omits a decisive memory step. During training, we use evidence-level supervision and rationale distillation to transfer the required evidence judgment into the lightweight router. At inference time, the router reads only (q, C_f) and produces a single-token termination decision.

Evidence-Sufficiency Data Construction

Training the router requires supervision that distinguishes evidence sufficiency from semantic relevance (as shown in fig. 2). We collect long-horizon agent trajectories from diverse interaction settings, including tool use, software engineering, web navigation, games, and long conversations. These training sources are disjoint from the downstream evaluation benchmarks, while preserving similar interaction patterns such as multi-step actions, environment feedback, and temporally dispersed evidence.

For each trajectory, we construct and verify a query–answer–evidence tuple (q_i, a_i, E_i) , where E_i contains the memory steps that support the reference answer a_i . We then apply evidence intervention to produce contexts with different sufficiency labels. A positive context retains all required evidence and mixes it with distractor steps:

$$C_i^+ = E_i \cup D_i, \quad (4)$$

where D_i contains randomly sampled memory steps from the same trajectory. Since the complete supporting evidence is preserved, C_i^+ is labeled as sufficient.

A negative context removes part or all of the required evidence and fills the remaining context with semantically related hard distractors:

$$C_i^- = (E_i \setminus E_i^{\text{drop}}) \cup D_i^{\text{hard}}. \quad (5)$$

The hard distractors may share entities, tools, or nearby events with the query, but do not preserve the complete evidence chain. These examples teach the router that relevance alone does not imply answerability.

Evidence removal does not always make a context insufficient, because redundant or alternative support may remain. We therefore evaluate each candidate negative with multiple answer models and discard it if any model still produces a correct answer. For every retained sample, a teacher model generates a rationale that explains which evidence is present and which decisive information is missing. The resulting dataset consists of tuples (q_i, C_i, y_i, r_i) , where $y_i \in \{\text{yes}, \text{no}\}$ is the sufficiency label and r_i is the teacher rationale. We split the data by trajectory to prevent related questions, neighboring steps, and context variants from leaking across training and validation sets.

Training Process

Given a constructed sample (q_i, C_i, y_i, r_i) , we form the student input as

$$x_i = \text{Prompt}(q_i, C_i). \quad (6)$$

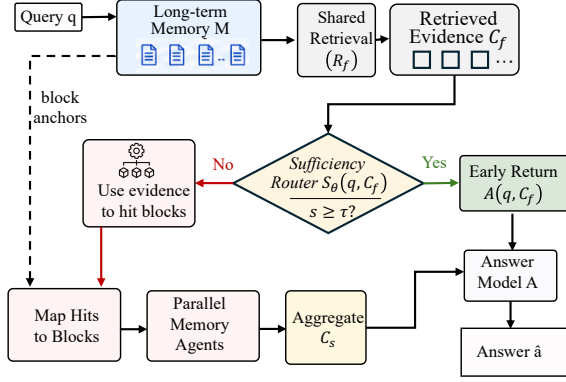


Figure 3: System Pipeline of Router-mem.

The router retains the causal language-model interface and predicts the sufficiency label through its next-token distribution. Let V_{yes} and V_{no} denote token groups corresponding to the normalized outputs “yes” and “no.” We optimize the grouped-token classification loss

$$\mathcal{L}_{\text{cls}}^i = -\log \sum_{v \in V_{y_i}} P_{\theta}(v | x_i). \quad (7)$$

The binary label specifies the desired decision but provides limited supervision about the underlying evidence judgment. We therefore use rationale-conditioned representation distillation. A frozen teacher additionally reads the rationale r_i , while the student observes only (q_i, C_i) . Let h_s^i denote the student representation before the decision token, and let h_t^i denote the teacher representation after processing the rationale. We align them at a selected intermediate layer using

$$\mathcal{L}_{\text{rep}}^i = 1 - \frac{(h_s^i)^\top h_t^i}{\|h_s^i\|_2 \|h_t^i\|_2}. \quad (8)$$

The final training objective is

$$\mathcal{L}_{\text{router}} = \mathcal{L}_{\text{cls}} + \lambda_{\text{rep}} s_0 \mathcal{L}_{\text{rep}}, \quad (9)$$

where λ_{rep} controls the contribution of representation distillation and s_0 calibrates the scales of the two losses. The teacher and rationale are used only during training. At inference time, the student reads (q, C_f) and produces a single decision token without generating an explanation.

Shared retriever training. We train the shared retriever with the same QA-evidence supervision. Evidence-complete memory views are treated as positives, while semantically related but incomplete views are used as negatives. Using an L2-normalized encoder $e(\cdot)$, we optimize a multi-positive contrastive objective:

$$\mathcal{L}_{\text{ret}}^i = -\frac{1}{|P_i|} \sum_{p \in P_i} \log \frac{\exp(e(q_i)^\top e(p)/T)}{\sum_{c \in C_i} \exp(e(q_i)^\top e(c)/T)}. \quad (10)$$

This objective encourages the shared prefix to retrieve answer-supporting evidence.

Algorithm 1: Progressive Memory Execution

Require: Query q , memory M , threshold τ

Ensure: Answer $\hat{a}$

```

1:  $(C_f, H_f) \leftarrow R_f(q, M)$   $\triangleright$  Shared retrieval prefix
2: if  $C_f$  fits the router input budget then
3:    $s \leftarrow s_{\theta}(q, C_f)$   $\triangleright$  Evidence-sufficiency score
4:   if  $s \geq \tau$  then
5:      $\hat{a} \leftarrow A(q, \text{Order}(C_f))$ 
6:     return  $\hat{a}$   $\triangleright$  Optimistic early return
7:   end if
8: end if
9:  $\mathcal{B}_f \leftarrow \text{MapToBlocks}(H_f)$ 
10:  $\mathcal{B}_s \leftarrow \text{SelectBlocks}(\mathcal{B}_f, M)$ 
11: for all  $b \in \mathcal{B}_s$  in parallel do
12:    $u_b \leftarrow \text{MemAgent}(q, b)$ 
13: end for
14:  $C_s \leftarrow \text{Aggregate}(\{u_b : b \in \mathcal{B}_s\})$ 
15:  $\hat{a} \leftarrow A(q, C_s)$ 
16: return  $\hat{a}$ 

```

Progressive Memory System

After training the retriever and termination router, we integrate them into a progressive memory execution system. The system first constructs aligned retrieval and processing views of the same memory (as shown in fig. 3). It then performs a shared retrieval step for every query and invokes broader memory processing only when the retrieved evidence is predicted to be insufficient.

Aligned memory views. Given an ordered memory $M = (z_1, \dots, z_T)$, we construct two aligned views. The retrieval view indexes individual memory steps and stores their corresponding step and block identifiers. The processing view groups temporally adjacent steps into overlapping memory blocks. This alignment allows each retrieval hit to directly locate the memory blocks used by subsequent processing. The two views therefore support different stages of one execution process, rather than two independent memory systems.

Shared retrieval and termination. For a query q , the shared retriever first returns an initial context C_f and its retrieval hits H_f :

$$C_f, H_f = R_f(q, M). \quad (11)$$

The router evaluates the sufficiency of C_f through its next-token distribution. We aggregate the probability assigned to the “yes” and “no” token groups:

$$p_{\text{yes}} = \sum_{v \in V_{\text{yes}}} P_{\theta}(v | q, C_f), \quad p_{\text{no}} = \sum_{v \in V_{\text{no}}} P_{\theta}(v | q, C_f), \quad (12)$$

and compute the normalized termination score

$$s(q, C_f) = \frac{p_{\text{yes}}}{p_{\text{yes}} + p_{\text{no}}}. \quad (13)$$

Given a threshold τ , the system follows

$$\pi_{\tau}(q, C_f) = \begin{cases} \text{terminate,} & s(q, C_f) \geq \tau, \\ \text{continue,} & s(q, C_f) < \tau. \end{cases} \quad (14)$$

Table 1: Results on AMA-Bench with two backbone LLMs: DeepSeek-V4-Flash and Qwen3.5-35B-A3B.

Method	DEEPSEEK-V4-FLASH					QWEN3.5-35B-A3B				
	Score (%)	Early (%)	T_{mem} (s)	T_{e2e} (s)	T_{build} (s)	Score (%)	Early (%)	T_{mem} (s)	T_{e2e} (s)	T_{build} (s)
<i>External Memory Baselines</i>										
RAG	40.46	–	0.057	2.49	10.15	45.21	–	0.059	4.72	9.88
LightMem	35.34	–	0.058	2.30	1157.78	36.58	–	0.055	2.49	6531.13
Mem0	35.90	–	0.159	3.50	46.12	43.96	–	0.156	2.61	51.37
Long Context	47.08	–	3.595	5.18	0.001	53.95	–	20.520	22.80	0.002
GAM	40.21	–	22.450	25.289	123.57	43.96	–	24.497	28.40	141.52
<i>Fixed Execution Endpoints</i>										
Fast Prefix Only	44.91	100.0	0.054	3.70	5.07	48.09	100.0	0.054	3.63	4.56
Full Completion (Slow)	56.49	0.0	11.341	14.47	5.07	56.41	0.0	15.816	20.28	4.66
<i>Router-Mem: Evidence-Conditioned Early Termination</i>										
Router-Mem ($\tau = 0.1$)	51.48	53.7	5.608	9.21	4.93	52.50	54.6	8.414	12.80	4.57
Router-Mem ($\tau = 0.3$)	52.68	38.5	7.215	10.53	4.90	53.65	39.2	9.351	13.10	4.65
Router-Mem ($\tau = 0.5$)	55.17	30.0	8.248	11.57	5.08	55.06	30.2	10.581	14.49	4.60
Router-Mem ($\tau = 0.7$)	54.73	22.1	9.045	12.26	4.98	55.52	22.3	13.888	18.67	4.71
Router-Mem ($\tau = 0.9$)	55.89	12.3	10.273	13.51	4.94	55.52	12.5	14.423	18.96	4.61

The threshold is selected on development data and fixed during evaluation. Varying τ yields different empirical trade-offs between answer quality and execution cost. If the retrieved context exceeds the router input budget, the system conservatively selects continuation.

Early return and execution continuation. If the termination condition is satisfied, the retrieved steps are restored to their original temporal order and passed to the answer model:

$$\hat{a}_f = A(q, C_f). \quad (15)$$

This branch is an early return from the full memory execution, rather than the output of an independently selected fast system.

Otherwise, the system reuses the retrieval hits to continue the same execution process. Let $\mathcal{B}_f$ denote the blocks associated with H_f . We expand these anchors into a bounded set of processing blocks:

$$\mathcal{B}_s = \text{SelectBlocks}(\mathcal{B}_f, M). \quad (16)$$

The selected blocks include the retrieved anchor regions and their temporal neighborhoods, which helps recover local action–feedback and cross-step dependencies. Each block is analyzed independently by a memory agent:

$$u_b = \text{MemAgent}(q, b), \quad b \in \mathcal{B}_s. \quad (17)$$

An aggregator combines the block-level results into a consolidated context:

$$C_s = \text{Aggregate}(\{u_b : b \in \mathcal{B}_s\}), \quad \hat{a}_s = A(q, C_s). \quad (18)$$

The continuation stage therefore preserves and extends the computation performed by the shared retrieval prefix. It is invoked only when the current evidence does not support early termination.

Experiment

We evaluate Router-Mem against state-of-the-art memory systems for LLM agents, focusing on effectiveness, routing

efficiency, and robustness. Our experiments investigate three key research questions:

RQ1: Comparative Effectiveness. Does Router-Mem provide advantages over existing memory mechanisms in terms of answer quality, inference latency, and overall efficiency?

RQ2: Quality–Latency Trade-off. Can Router-Mem achieve a favorable balance between memory answer performance and inference latency, and how does the routing threshold control this trade-off?

RQ3: Training Effectiveness. Do the proposed router training and embedding training improve retrieval quality, routing decisions, and end-to-end system performance?

Experimental Setup.

Dataset. We evaluate Router-Mem on two representative long-term memory benchmarks for LLM agents: AMA-Bench (Zhao et al. 2026) and BEAM (Tavakoli et al. 2026). AMA-Bench is designed to evaluate memory capabilities in long-horizon agentic scenarios, covering diverse interactions where agents must retrieve and reason over previously observed experiences. It emphasizes accurate memory access under complex multi-step dependencies. BEAM focuses on extremely long-context memory understanding and evaluates whether agents can effectively recover relevant information from large-scale conversational histories. Together, these benchmarks cover complementary challenges: AMA-Bench tests structured agent memory reasoning, while BEAM stresses scalability under massive context lengths. They provide a comprehensive evaluation of both memory effectiveness and efficient execution.

Settings. We compare Router-Mem with representative memory baselines, including standard RAG (Lewis et al. 2020), compact memory systems such as LightMem (Fang et al. 2026) and Mem0 (Chhikara et al. 2025), Long Context, and deep query-time memory reconstruction methods

Table 2: End-to-end results on BEAM with Model DeepSeek-V4-Flash.

Method	Score (%)	Early Term. (%)	T_{mem} (s)	T_{e2e} (s)	T_{build} (s)
<i>External Memory Baselines</i>					
RAG	30.29	—	0.094	1.66	217.01
LightMem	27.12	—	0.058	1.55	1174.31
Mem0	22.09	—	0.151	1.75	982.20
Long Context	38.01	—	14.304	17.08	0.05
GAM	35.23	—	25.817	28.435	1334.78
<i>Fixed Execution Endpoints</i>					
Fast Prefix Only	30.02	100.0	0.054	2.51	31.81
Full Completion (Slow)	43.26	0.0	14.611	18.43	31.90
<i>Router-Mem: Evidence-Conditioned Early Termination</i>					
Router-Mem ($\tau = 0.1$)	33.94	56.9	5.675	8.51	32.21
Router-Mem ($\tau = 0.3$)	36.53	39.4	8.781	11.72	31.84
Router-Mem ($\tau = 0.5$)	38.77	26.6	10.883	14.11	31.88
Router-Mem ($\tau = 0.7$)	39.81	16.7	12.512	16.18	32.01
Router-Mem ($\tau = 0.9$)	42.09	8.0	13.422	16.93	31.86

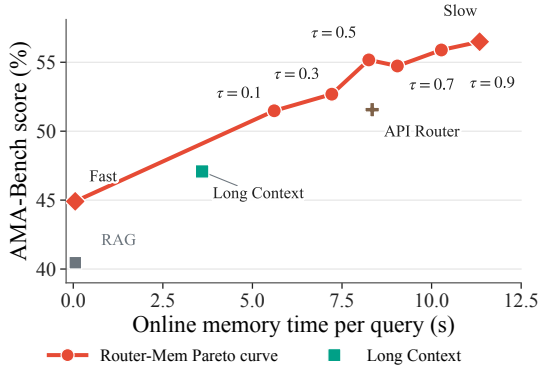


Figure 4: Quality-latency under different routing thresholds.

GAM (Yan et al. 2025). For evaluation, we use DeepSeek-V4-Flash as the backbone model and report answer quality measured by the official LLM judge provided by each benchmark. We evaluate both efficiency and effectiveness by reporting memory construction time, retrieval latency, and end-to-end inference latency. Further experiments analyze the impact of routing thresholds τ , router training components, and embedding model choices to evaluate the robustness and scalability of Router-Mem. We also evaluate the E2E token cost of Router-mem against existing agentic memory baselines in **technical supplement**.

Main Results.

We compare Router-Mem with representative memory mechanisms from three categories. RAG serves as a lightweight retrieval baseline that directly retrieves relevant contexts without persistent memory modeling. LightMem and Mem0 represent compact memory systems that extract and organize historical information into efficient memory representations. Long Context provides a full-context baseline that directly exposes the entire history to the language model. We further compare against two execution endpoints: Fast Prefix Only,

which always terminates after the shared retrieval stage, and Full Completion (Slow), which always performs complete memory reconstruction. These endpoints characterize the lower-cost and upper-performance boundaries of progressive memory execution.

Results on AMA-Bench. Table 1 shows that Router-Mem consistently achieves a favorable quality-latency trade-off between the fast and slow endpoints. The Fast Prefix Only baseline achieves the lowest memory latency (0.054s) but suffers from incomplete evidence coverage, obtaining 44.91% score. In contrast, Full Completion reaches the highest score of 56.49% with substantially larger latency. By adaptively routing queries, Router-Mem approaches the performance of full completion while avoiding unnecessary deep memory processing. For example, Router-Mem ($\tau = 0.5$) achieves 55.17% score with only 8.25s memory latency, reducing the cost by 27.3% compared with full completion. Different thresholds provide controllable operating points, demonstrating that the learned router effectively balances answer quality and execution efficiency.

Results on BEAM. Results on BEAM further demonstrate the scalability of Router-Mem under extremely long conversational histories. The Fast Prefix Only baseline obtains only 30.02% score, while Full Completion improves performance to 43.26% at the cost of 14.61s memory latency. Router-Mem bridges this gap by allocating deeper reasoning only when necessary. With $\tau = 0.7$, it achieves 39.81% score while reducing memory latency to 12.51s, and with $\tau = 0.9$, it approaches the full completion performance (42.09%) with lower inference cost. Across thresholds, Router-Mem forms a smooth quality-cost frontier, validating that evidence-conditioned routing enables adaptive computation allocation for long-term agent memory.

Ablation Study

Memory Performance and Latency Trade-off. Figure 4 illustrates how the routing threshold controls the quality-latency trade-off on AMA-Bench. Increasing τ makes the

Table 3: Router training comparison on AMA-Bench and BEAM. ET denotes the early-termination rate.

τ	Router	AMA-Bench			BEAM		
		Score (%)	ET (%)	T_{e2e} (s)	Score (%)	ET (%)	T_{e2e} (s)
0.3	Base	55.63	8.0	13.68	29.71	90.5	4.15
	Trained	52.68	38.5	10.53	36.53	39.4	11.72
0.5	Base	55.52	4.9	14.21	29.62	86.5	5.15
	Trained	55.17	30.0	11.57	38.77	26.6	14.11
0.7	Base	55.79	2.2	14.39	31.78	80.5	5.67
	Trained	54.73	22.1	12.26	39.81	16.7	16.18

Table 4: Effect of embedding training on AMA-Bench.

Setting	Embedding	Score (%)	Fast hit (%)	T_{mem} (s)	T_{e2e} (s)
Fast-only	Base	42.00	100.0	0.055	3.41
Fast-only	Trained	44.91	100.0	0.054	3.70
Router ($\tau = 0.5$)	Base	54.79	21.8	9.17	12.16
Router ($\tau = 0.5$)	Trained	55.17	30.0	8.25	11.57
Slow-only	Base	55.73	0.0	11.29	14.32
Slow-only	Trained	56.49	0.0	11.34	14.47

router more conservative and sends more queries to the deeper memory-processing path. This generally improves answer quality at the cost of additional online memory time. Among the resulting operating points, $\tau = 0.5$ provides a particularly favorable Pareto-efficient trade-off. At a comparable online memory time, this operating point also achieves a substantially higher AMA-Bench score than the large model API Router baseline. It achieves a score of 55.17%, only 1.32 points below the Full Completion endpoint (56.49%), while reducing T_{mem} from 11.341s to 8.248s and T_{e2e} from 14.47s to 11.57s. These correspond to 27.3% and 20.0% latency reductions, respectively. Thus, $\tau = 0.5$ retains most of the accuracy benefit of full memory reconstruction while avoiding a substantial fraction of its computational overhead.

Effect of Router Training. As shown in Table 3, router training primarily calibrates the fast-slow decision boundary rather than uniformly favoring one execution path. On AMA-Bench, the base router is strongly biased toward the slow path, with only 4.9% early termination at $\tau = 0.5$. Training raises this rate to 30.0% and reduces T_{e2e} from 14.21s to 11.57s, while preserving a comparable score (55.52% vs. 55.17%). In contrast, the base router on BEAM is over-confident in the fast path, terminating 86.5% of queries early and achieving only 29.62%. Training corrects this bias, reducing early termination to 26.6% and improving the score to 38.77%. Moreover, the trained router responds consistently to changes in τ , enabling flexible control over computation. Without training, the decision boundary collapses toward opposite fixed endpoints across the two benchmarks, making threshold adjustment largely ineffective.

Effect of Embedding Training. We study the effect of training the embedding model on AMA-Bench. As shown in Table 4, the trained embedding improves the Fast-only score from 42.00 to 44.91 without changing the memory la-

Table 5: Effect of jointly training the router and retriever on AMA-Bench.

τ	Training	Score (%)	Early Term. (%)	T_{mem} (s)	T_{e2e} (s)
0.3	None	54.48	9.4	10.352	13.453
	Joint	52.68	38.5	7.215	10.534
0.5	None	53.96	6.1	10.842	13.987
	Joint	55.17	30.0	8.248	11.570
0.7	None	56.25	3.5	11.073	14.219
	Joint	54.73	22.1	9.045	12.260

tency. Under the practical Router-Mem setting at $\tau = 0.5$, it increases the score from 54.79 to 55.17 and the fast-hit rate from 21.8% to 30.0%, while reducing T_{mem} from 9.17s to 8.25s and T_{e2e} from 12.16s to 11.57s. The Slow-only endpoint also gains 0.76 points with nearly unchanged runtime. These results indicate that embedding training primarily improves retrieved evidence quality and routing efficiency rather than increasing computation.

Effect of Joint Training. We also evaluate the joint effect of training both the router and retriever. As shown in Table 5, without training, the system behaves similarly to the Slow-only endpoint: early termination remains between 3.5% and 9.4%, and varying τ provides limited control over computation. Joint training increases this range to 22.1%–38.5% and consistently reduces both memory and end-to-end latency while preserving comparable answer quality. At $\tau = 0.5$, it improves the score from 53.96% to 55.17%, raises early termination from 6.1% to 30.0%, and reduces T_{mem} from 10.842s to 8.248s. This broader operating range indicates that the learned components jointly calibrate retrieval quality and termination confidence across thresholds. These results show that joint training more reliably identifies safely answerable queries and makes τ an effective control knob for the quality-latency trade-off.

Conclusion

In this paper, we propose Router-Mem to address the challenge of efficiently accessing long-horizon memories in LLM agents, where existing methods often trade off between low-cost retrieval and expensive deep memory reasoning. Router-Mem is an evidence-conditioned progressive memory execution framework that learns when the current retrieved context is sufficient to terminate further computation. It first performs a shared retrieval prefix and employs a lightweight sufficiency router to make an early-termination decision. In particular, the shared prefix couples the fast and slow paths, allowing the system to preserve retrieved evidence and avoid restarting memory search when deeper processing is required. When evidence is insufficient, it reuses retrieval anchors to guide memory expansion and deeper evidence aggregation. Experiments on AMA-Bench and BEAM demonstrate that Router-Mem achieves strong answer quality while substantially reducing inference cost. We believe our approach provides a practical direction toward adaptive memory systems that allocate reasoning resources according to the actual evidence requirements of each query.

References

- Bai, Y.; Lv, X.; Zhang, J.; Lyu, H.; Tang, J.; Huang, Z.; Du, Z.; Liu, X.; Zeng, A.; Hou, L.; Dong, Y.; Tang, J.; and Li, J. 2024. LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 3119–3137.
- Beltagy, I.; Peters, M. E.; and Cohan, A. 2020. Longformer: The Long-Document Transformer. *arXiv preprint arXiv:2004.05150*.
- Chhikara, P.; Khant, D.; Aryan, S.; Singh, T.; and Yadav, D. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv preprint arXiv:2504.19413*.
- Dai, Z.; Yang, Z.; Yang, Y.; Carbonell, J.; Le, Q.; and Salakhutdinov, R. 2019. Transformer-XL: Attentive Language Models beyond a Fixed-Length Context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2978–2988.
- Dao, T.; Fu, D. Y.; Ermon, S.; Rudra, A.; and Ré, C. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems*.
- Du, P. 2026. Memory for autonomous llm agents: Mechanisms, evaluation, and emerging frontiers. *arXiv preprint arXiv:2603.07670*.
- Edge, D.; Trinh, H.; Cheng, N.; Bradley, J.; Chao, A.; Mody, A.; Truitt, S.; Metropolitansky, D.; Ness, R. O.; and Larson, J. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130*.
- Fang, J.; Deng, X.; Xu, H.; Jiang, Z.; Tang, Y.; Xu, Z.; Deng, S.; Yao, Y.; Wang, M.; Qiao, S.; Chen, H.; and Zhang, N. 2026. LightMem: Lightweight and Efficient Memory-Augmented Generation. *arXiv:2510.18866*.
- Guo, T.; Chen, X.; Wang, Y.; Chang, R.; Pei, S.; Chawla, N. V.; Wiest, O.; and Zhang, X. 2024a. Large language model based multi-agents: A survey of progress and challenges. *arXiv preprint arXiv:2402.01680*.
- Guo, Z.; Xia, L.; Yu, Y.; Ao, T.; and Huang, C. 2024b. Ligh-tRAG: Simple and Fast Retrieval-Augmented Generation. *arXiv preprint arXiv:2410.05779*.
- Gutiérrez, B. J.; Shu, Y.; Gu, Y.; Yasunaga, M.; and Su, Y. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. In *Advances in Neural Information Processing Systems*.
- Hsieh, C.-P.; Sun, S.; Krizan, S.; Acharya, S.; Rekesh, D.; Jia, F.; Zhang, Y.; and Ginsburg, B. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models? In *Conference on Language Modeling*.
- Hu, Y.; Liu, S.; Yue, Y.; Zhang, G.; Liu, B.; Zhu, F.; Lin, J.; Guo, H.; Dou, S.; Xi, Z.; et al. 2025. Memory in the age of ai agents. *arXiv preprint arXiv:2512.13564*.
- Jeong, S.; Baek, J.; Cho, S.; Hwang, S. J.; and Park, J. 2024. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 7036–7050. Association for Computational Linguistics.
- Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-t.; Rocktäschel, T.; Riedel, S.; and Kiela, D. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*.
- Li, X.; Wang, S.; Zeng, S.; Wu, Y.; and Yang, Y. 2024. A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinearth*, 1(1): 9.
- Li, Y.; Yan, Z.; and Käfer, T. 2026. RASER: Recoverability-Aware Selective Escalation Router for Multi-Hop Question Answering. *arXiv preprint arXiv:2606.02488*.
- Liu, N. F.; Lin, K.; Hewitt, J.; Paranjape, A.; Bevilacqua, M.; Petroni, F.; and Liang, P. 2024a. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*, 12: 157–173.
- Liu, X.; Yu, H.; Zhang, H.; Xu, Y.; Lei, X.; Lai, H.; Gu, Y.; Ding, H.; Men, K.; Yang, K.; Zhang, S.; Deng, X.; Zeng, A.; Du, Z.; Zhang, C.; Shen, S.; Zhang, T.; Su, Y.; Sun, H.; Huang, M.; Dong, Y.; and Tang, J. 2024b. AgentBench: Evaluating LLMs as Agents. In *International Conference on Learning Representations*.
- Ouyang, S.; Yan, J.; Hsu, I.-H.; Chen, Y.; Jiang, K.; Wang, Z.; Han, R.; Le, L. T.; Daruki, S.; Tang, X.; Tirumalashetty, V.; Lee, G.; Rofouei, M.; Lin, H.; Han, J.; Lee, C.-Y.; and Pfister, T. 2026. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory. In *International Conference on Learning Representations*.
- Park, J.; Cho, S.; and Lee, J.-Y. 2025. Stop-RAG: Value-Based Retrieval Control for Iterative RAG. *arXiv preprint arXiv:2510.14337*.
- Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Advances in Neural Information Processing Systems*.
- Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*.
- Tavakoli, M.; Salemi, A.; Ye, C.; Abdalla, M.; Zamani, H.; and Mitchell, J. R. 2026. Beyond a Million Tokens: Benchmarking and Enhancing Long-Term Memory in LLMs. In *International Conference on Learning Representations*.
- Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*.
- Wang, K.; Lin, Y.; Lou, J.; Zhou, Z.; Suvonov, B.; and Li, J. 2026. E-mem: Multi-agent based Episodic Context Reconstruction for LLM Agent Memory. *arXiv:2601.21714*.
- Xu, W.; Liang, Z.; Mei, K.; Gao, H.; Tan, J.; and Zhang, Y. 2025. A-MEM: Agentic Memory for LLM Agents. In *Advances in Neural Information Processing Systems*.

Yan, B. Y.; Li, C.; Qian, H.; Lu, S.; and Liu, Z. 2025. General Agentic Memory Via Deep Research. *arXiv preprint arXiv:2511.18423*.

Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*.

Zaheer, M.; Guruganesh, G.; Dubey, A.; Ainslie, J.; Alberti, C.; Ontañón, S.; Pham, P.; Ravula, A.; Wang, Q.; Yang, L.; and Ahmed, A. 2020. Big Bird: Transformers for Longer Sequences. In *Advances in Neural Information Processing Systems*.

Zhang, G.; Fu, M.; Wan, G.; Yu, M.; Wang, K.; and Yan, S. 2025a. G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems. *arXiv preprint arXiv:2506.07398*.

Zhang, X.; Chen, Y.; Hu, S.; Xu, Z.; Chen, J.; Hao, M. K.; Han, X.; Thai, Z. L.; Wang, S.; Liu, Z.; and Sun, M. 2024. ∞ Bench: Extending Long Context Evaluation Beyond 100K Tokens. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*.

Zhang, Z.; Dai, Q.; Bo, X.; Ma, C.; Li, R.; Chen, X.; Zhu, J.; Dong, Z.; and Wen, J.-R. 2025b. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43(6): 1–47.

Zhao, Y.; Yuan, B.; Huang, J.; Yuan, H.; Yu, Z.; Xu, H.; Hu, L.; Shankarampeta, A.; Huang, Z.; Ni, W.; Tian, Y.; and Zhao, J. 2026. AMA-Bench: Evaluating Long-Horizon Memory for Agentic Applications. In *Proceedings of the 43rd International Conference on Machine Learning*.

Zhou, S.; Xu, F. F.; Zhu, H.; Zhou, X.; Lo, R.; Sridhar, A.; Cheng, X.; Ou, T.; Bisk, Y.; Fried, D.; Alon, U.; and Neubig, G. 2024. WebArena: A Realistic Web Environment for Building Autonomous Agents. In *International Conference on Learning Representations*.